

字节域语义内容理解实验说明

助教：李方成

lifangcheng2002@163.com

教师：吴科君

kjwu@hust.edu.cn

01 多媒体通信与码流损坏

图像和视频经过压缩编码后，以二进制码流的形式传输、存储和解析。



像素、压缩文件和字节序列

同一幅图像可以有不同的表示：

- 像素矩阵：适合传统视觉网络
- JPEG 文件：包含标记、表和扫描数据
- 字节序列：统一处理图像、视频和其他文件

码流损坏为什么值得研究



码流模型尝试跳过不可靠的解码过程，
直接从字节中提取内容特征。

02 JPEG：从文件标记到扫描数据

JPEG 不是一串无意义的数字；不同段落共同描述图像如何被解码。

JPEG 文件从标记段到扫描数据按顺序组织



示例字节（十六进制）

FF D8 FF DB 00 43 00 06 04 04 05 04 04 FF C0 00 11 08 00 1C 00 1C FF DA ...

标记字节帮助解码器定位结构；扫描数据保存真正的压缩图像内容。

SOI / EOI 标记文件边界；DQT 和 DHT 保存解码所需的表；SOS 之后是压缩扫描数据。
标记段或扫描数据受损，可能导致解码失败、错块或花屏。

03 H.264：参数、关键帧与预测帧

视频码流由多个 NAL 单元组成，帧之间还存在时间上的依赖关系。

H.264 将视频切成多个 NAL 单元，参数和图像帧分开保存



SPS/PPS 受损：后续帧可能无法正确解析；IDR 受损：预测链条可能受到影响。

SPS / PPS 决定后续帧如何解析；IDR 是可以独立解码的关键帧；P、B 帧依赖参考帧。一处错误可能沿着后续帧传播，这也是视频码流分析比单张图更复杂的原因。

04 突发损坏： Gilbert-Elliott 二状态模型

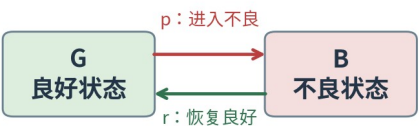
错误不一定独立均匀出现；网络拥塞、存储故障等因素会让错误连续发生。

模型的两个状态

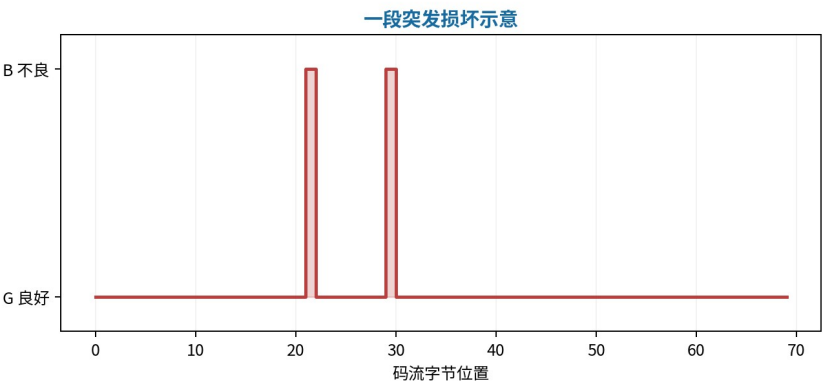
G：良好状态，错误率低

B：不良状态，错误率高

Gilbert-Elliott 二状态模型：用状态转移描述突发损坏



r 较小时，B 状态会持续更久，形成突发错误。



p ： $G \rightarrow B$ 的转移概率

r ： $B \rightarrow G$ 的恢复概率

r 越小，突发错误持续越久。

05 码流损坏的影响：为什么需要字节域理解

损坏位置不同，解码器可能直接失败，也可能输出不可靠的像素或后续帧。

损坏位置 → 解码结果 → 对视觉模型的影响

文件头 / 参数表

JPEG 的 SOI、DQT、DHT ;
H.264 的 SPS、PPS

无法解析文件或后续帧

扫描数据 / 关键帧

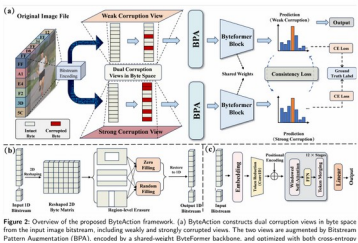
JPEG 压缩数据;
H.264 的 IDR、P、B 帧

错块、花屏、错误传播

解码输出

像素缺失、尺寸异常或
整段视频无法恢复

像素域模型没有可靠输入



传统流程：码流 → 解码 → 像素 → 视觉模型

字节域流程：码流 → 字节模型 → 分类或语义理解

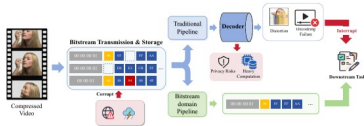


Fig. 1: Comparison between conventional pixel-based video processing and the proposed bitstream-based method.

06 代表性字节模型

字节模型直接读取 0—255 字节值，重点解决超长序列和多尺度结构建模。

MEGABYTE

2023

全局模型处理字节块
局部模型预测块内字节
层级生成

ByteFormer

2023

卷积缩短序列窗口
Transformer 分类
文件分类

MambaByte

2024

选择性状态空间模型
近似线性处理长序列
长序列建模

bGPT

2024

Patch-level + Byte-level
多模态字节生成
通用字节生成

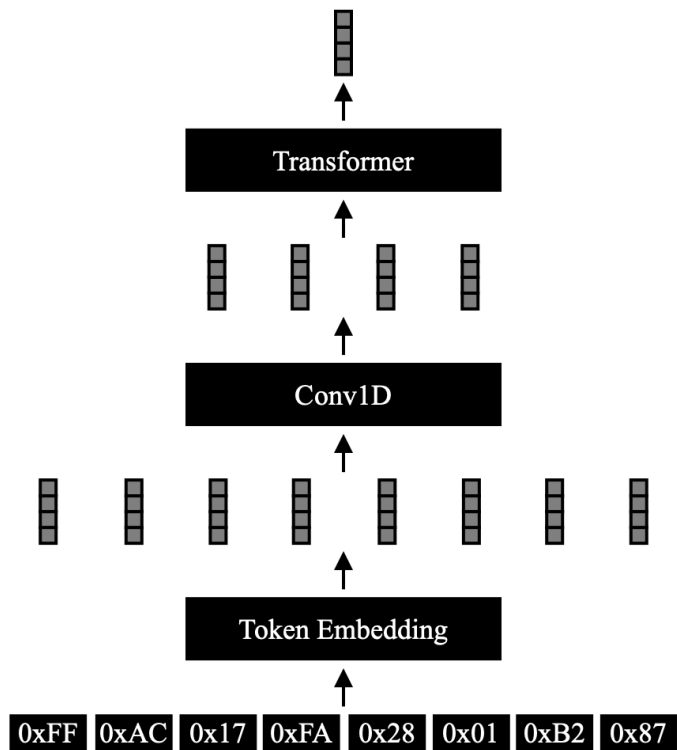
mBLM

2025

多层次 Byte Patch
Transformer / Mamba 可选
百万字节上下文

课程选择 ByteFormer：面向文件分类，有公开代码和预训练权重，适合完成 MNIST 微调。

07 ByteFormer 框架与课程任务

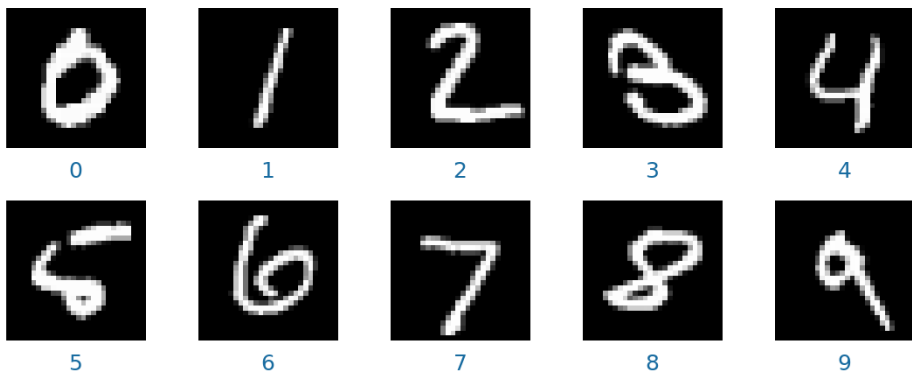


- 使用公开的 ByteFormer Tiny 预训练权重
- 将分类头改为 10 个类别
- 在训练集更新参数，用验证集选择模型
- 最后在独立测试集上报告结果

基础任务：完成码流图像分类、参数对比和结果分析。

08 课程数据集与三类划分

课程仓库已经提供固定划分，所有同学使用相同样本。



训练集

5,000

每类 500 张
包含原图与轻微
旋转、平移视图

验证集

1,000

每类 100 张
用于选择最佳模
型

测试集

1,000

每类 100 张
训练结束后独立
评估

数据文件: data/course_1of10/ 测试集不参与训练和模型选择。

09 实验环境与在线平台

平台界面、免费额度和租用价格可能调整，以平台当前页面为准。

运行平台	适用情况	入口
Kaggle 免费 GPU	推荐；导入课程 Notebook 后运行	Notebook + Internet + GPU
AutoDL 租用 GPU	Kaggle 额度不足时使用	选择 PyTorch 镜像，打开 Terminal
本地 GPU	已安装 Python 3.9—3.12	<code>pip install -r requirements-local.txt</code>

[知乎 | Kaggle GPU资源使用教程](#)

[CSDN | Kaggle平台使用指导](#)

[Kaggle官方Notebook文档](#)

[AutoDL官方快速开始](#)

10 获取代码、数据与预训练权重

课程 Notebook 中已经写好相同命令。

```
!git clone https://github.com/Franklin-L/byteformer-mnist-course.git
%cd /kaggle/working/byteformer-mnist-course
!python -m pip install -r requirements.txt
!python prepare.py
```

看到 [READY] 即准备完成；课程数据集随仓库提供， ByteFormer 预训练权重自动下载并校验。

train_course_subset.py : 训练 evaluate_course_corruption.py : 测试
predict.py : 单图预测 data/course_1of10/ : 固定数据 outputs/ : 实验结果

11 模型训练与参数设置

训练 / 验证 / 测试 5,000 / 1,000 / 1,000

训练轮数 根据验证结果自行设置

batch size 自行设置，参考 32

学习率 主干 $1e-4$ ；分类头 10 倍

输入增强 轻微旋转与水平平移

```
EPOCHS = int(input("epochs: "))
BATCH_SIZE = int(input("batch: "))
!python train_course_subset.py \
  --method clean --epochs {EPOCHS} \
  --batch-size {BATCH_SIZE} --clean-augmentations \
  --output outputs/course_clean
```

训练过程中观察验证集准确率；
结果保存在 outputs/course_clean/。

轮数与 batch size 没有固定答案，请在报告中写明自己的设置。

12 训练结果分析

```
from IPython.display import display, Image
display(Image("outputs/course_clean/curves.png"))
```

训练损失

是否总体下降？
是否出现明显波动？

验证结果

准确率怎样变化？
最佳结果出现在哪一轮？

测试结果

加载验证集选出的模型，
在独立测试集上评估。

history.csv 保存每轮结果； metrics.json 保存最佳轮次、参数和运行时间。

13 测试、单图预测与错例检查

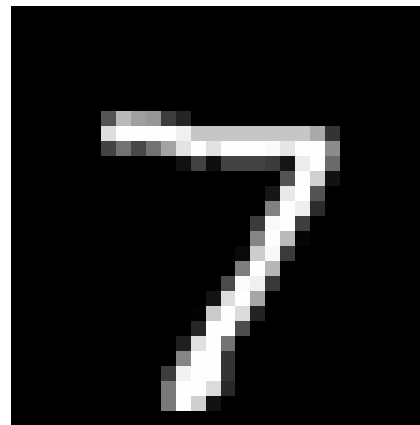
```
!python evaluate_course_corruption.py \  
  --checkpoint outputs/course_clean/best.pt --output outputs/course_clean_eval  
!python predict.py --checkpoint outputs/course_clean/best.pt --index 0
```

Clean：基础任务的独立测试结果。

Medium-Flip / Loss / Mixed：加分项使用。

将 --index 改成其他测试索引，可查看预测概率。

结合预测图和错误索引分析典型错例。



真实标签 7 | 模型预测 7

14 参数对比实验

```
COMPARISON_EPOCHS = int(input("epochs: "))  
COMPARISON_BATCH_SIZE = int(input("batch: "))  
!python train_course_subset.py --method clean \  
  --epochs {COMPARISON_EPOCHS} --batch-size {COMPARISON_BATCH_SIZE} \  
  --clean-augmentations --output outputs/comparison
```

轮数和 batch size 均可自行设置。

建议一次只改变一个参数，其他条件保持一致。

保留第一次结果，另存 outputs/comparison。

比较内容

- 验证集准确率与训练时间
- 损失曲线和收敛速度
- 参数变化带来的收益与代价

报告中列出两次设置及结果，并说明比较结论。

15 AutoDL 运行与常见问题

```
cd /root/autodl-tmp
git clone https://github.com/Franklin-L/byteformer-mnist-course.git
cd byteformer-mnist-course
python -m pip install -r requirements.txt
python prepare.py
python train_course_subset.py --method clean \
    --epochs "$EPOCHS" --batch-size "$BATCH_SIZE" \
    --clean-augmentations --output outputs/course_clean
```

运行前在 Terminal 中设置 EPOCHS 和 BATCH_SIZE ;
实验结束后下载 outputs 并在控制台关机。

没有使用 GPU 检查 Kaggle 加速器或 AutoDL 实例

缺少 Python 包 重新安装 requirements.txt

显存不足 减小 batch size 并记录实际值

输出目录已有文件 改用新的 output 目录

[AutoDL官方快速开始](#)

16 实验结果与报告提交

- ① 保存训练曲线和测试结果。
- ② 完成一次参数对比。
- ③ 展示单图预测或典型错例。
- ④ 填写实验环境、命令、参数与结果分析。
- ⑤ 打包代码、结果图和实验报告。

数据集和 Python 环境不需要重复提交。

基础任务检查

完成训练、验证和测试流程

记录实际轮数与 batch size

区分验证集结果和测试集结果

说明曲线变化与参数对比

给出预测结果与错例分析

提交个人实验报告

报告分别列出训练集、验证集和测试集的用途与结果。

17 加分项：损坏码流分类

课程提供与干净测试集对应的损坏测试数据，共包含以下两种基本损坏。

Bit flip | 比特翻转

翻转选中字节的 1 位（长度不变）

10100110 → 10100010

Byte loss | 字节丢失

删除字节，后续字节前移

FF D8 A1 3C → FF D8 3C

Mixed | 混合损坏

每个样本随机选择翻转或丢失
检验模型对未知损坏的适应能力

可以尝试的方向

① 结果分析

比较 Clean、Flip、Loss、Mixed，
观察哪类损坏影响更大。

② 损坏增强

训练时随机加入 bit flip 和 byte loss 样本。

③ 一致性约束

让同一图像的干净码流与损坏码流输出接近。

参考资料：[CBSU-ALLM](#) [BRACE](#) [BSCV](#)

提交改进方法、训练设置以及四类测试结果；可自行组合其他增强或鲁棒训练方法。

18 配套材料与参考文献

实验步骤、命令和数据说明见课程仓库与 Notebook。

[课程仓库：代码、Notebook、数据、PPT和报告模板](#)

[ByteFormer: Bytes Are All You Need](#)

[MEGABYTE: Predicting Million-byte Sequences](#)

[MambaByte: Token-free Selective State Space Model](#)

[bGPT: Byte Models are Digital World Simulators](#)

[mBLM: Hierarchical Transformers for Byte Modeling](#)

[Kaggle Notebook官方文档](#)

[AutoDL官方快速开始](#)